HDOC Day-48

Pivot Index of a Variable-Length Array Using Pointers

Problem Statement

Read a variable-length integer array and pass its address to pivot_index(). Find the leftmost zero-based index whose strictly-left sum equals its strictly-right sum; otherwise display -1.

Algorithm

1. Read a positive array size and declare a variable-length integer array.
2. Read all integer elements into the array.
3. Pass the address of the array and its size to pivot_index().
4. Compute total sum of all elements. Set left sum to zero.
5. For each index i from 0 to size - 1, compute right sum = total - left sum - arr[i].
6. If left sum equals right sum, display index i and return immediately (leftmost pivot).
7. Otherwise add arr[i] to the left sum and continue.
8. If no index satisfies the condition, display -1.

Test Case Table

TC	Size	Array input	Expected pivot	Purpose
1	6	1 7 3 6 5 6	3	Example 1: interior pivot
2	3	1 2 3	-1	Example 2: no pivot
3	3	2 1 -1	0	Example 3: pivot at first index
4	1	9	0	Single element
5	5	0 0 0 0 0	0	Multiple pivots; choose leftmost
6	3	1 2 1	1	Pivot in middle
7	4	1 2 3 4	-1	No pivot
8	0	Not applicable	Invalid size	Non-positive size

C Program

```
#include <stdio.h>
#include <stdlib.h>

void pivot_index(const int *arr, int size)
{
    long long total = 0, left = 0;
    int i;
    for (i = 0; i < size; i++) total += *(arr + i);
    for (i = 0; i < size; i++) {
        long long right = total - left - *(arr + i);
        if (left == right) {
            printf("Pivot index: %d\n", i);
            return;
        }
        left += *(arr + i);
    }
```

```
    printf("Pivot index: -1\n");
}

int main(void)
{
    int size, i;
    printf("Enter array size: ");
    if (scanf("%d", &size) != 1 || size <= 0) {
        printf("Invalid size. Size must be positive.\n");
        return 1;
    }
    int arr[size];
    printf("Enter %d array elements:\n", size);
    for (i = 0; i < size; i++) {
        if (scanf("%d", arr + i) != 1) {
            printf("Invalid array element.\n");
            return 1;
        }
    }
    pivot_index(arr, size);
    return 0;
}
```

Compilation and Execution Commands

gcc hdoc48.c -o hdoc48
./hdoc48

Compilation and Execution

Paste a screenshot showing the gcc compilation command and successful execution.



Test Case 1

Input size: 6 | Array: 1 7 3 6 5 6 | Expected: 3. Example 1: interior pivot.



Test Case 2

Input size: 3 | Array: 1 2 3 | Expected: -1. Example 2: no pivot.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc48
Enter array size: 3
Enter 3 array elements:
1 2 3
Pivot index: -1
```

Test Case 3

Input size: 3 | Array: 2 1 -1 | Expected: 0. Example 3: pivot at first index.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc48
Enter array size: 3
Enter 3 array elements:
2 1 -1
Pivot index: 0
```

Test Case 4

Input size: 1 | Array: 9 | Expected: 0. Single element.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc48
Enter array size: 1
Enter 1 array elements:
9
Pivot index: 0
```

Test Case 5

Input size: 5 | Array: 0 0 0 0 0 | Expected: 0. Multiple pivots; choose leftmost.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc48
Enter array size: 5
Enter 5 array elements:
0 0 0 0 0
Pivot index: 0
```

Test Case 6

Input size: 3 | Array: 1 2 1 | Expected: 1. Pivot in middle.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc48
Enter array size: 3
Enter 3 array elements:
1 2 1
Pivot index: 1
```

Test Case 7

Input size: 4 | Array: 1 2 3 4 | Expected: -1. No pivot.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc48
Enter array size: 4
Enter 4 array elements:
1 2 3 4
Pivot index: -1
```

Test Case 8

Input size: 0 | Array: Not applicable | Expected: Invalid size. Non-positive size.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc48
Enter array size: 0
Invalid size. Size must be positive.
```